COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE010/Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	October 13, 2026	**DATE SUBMITTED:**	October 13, 2026 - progress
NAME:	Dizor, Elijah Gresola, Aryanna Keith	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 12.1 Algorithmic Strategies

1. PROCEDURE OUTPUT

Table 12-1. Algorithmic Strategies and Examples

STRATEGY	ALGORITHM	ANALYSIS
Recursion	Merge Sort	Merge Sort uses recursion by repeatedly calling itself to divide an array into smaller parts until each part contains only one element. Once the array is divided, the smaller parts are sorted and combined until the entire array is arranged in the correct order.
Brute Force	Linear Search	Linear Search uses brute force by checking each element in an array one at a time until the target is found or all elements have been checked. It does not skip elements or divide the array into smaller parts before searching. Instead, it follows a direct approach that is simple to understand and implement, although it may take longer when the target is near the end or is not in the array.
Backtracking	Depth-First Search	DFS explores paths in a tree or graph by moving from one node to another and going as deep as possible along a path before exploring other options. It uses backtracking by returning to a previous node when it reaches a dead end or has no more unvisited nodes to explore. From there, it continues along another available path.
Greedy	Selection Sort	Selection Sort follows a greedy approach by repeatedly choosing the smallest element from the unsorted portion of an array and placing it in its correct position. At each step, it selects the best available choice without reconsidering previous selections. This process continues until all elements are arranged in ascending order.
Divide-and-Conquer	Merge Sort	Merge Sort uses divide-and-conquer by repeatedly dividing an array into smaller parts until each part contains only one element. It then sorts and merges these parts step by step until the entire array is arranged in the correct order. This makes a large problem easier to solve by working on smaller parts first.

Table 12-2. Memoization Implementation

INPUT

memoization.cpp > ...

```cpp
#include <iostream>
#include <algorithm>

int memo[1001];

int getMinSteps(int n){
    if (n == 1) return 0;
    if (memo[n] != -1) return memo[n];

    int r = 1 + getMinSteps(n - 1);

    if (n % 2 == 0)
        r = std::min(r, 1 + getMinSteps (n / 2));

    if (n % 3 == 0)
        r = std::min(r, 1 + getMinSteps (n / 3));

    memo[n] = r;
    return r;
}

int main (){
    int n;

    std::cout << "--- MEMOIZATION ---" << std::endl;
    std::cout << "Enter a positive integer: ";
    std::cin >> n;

    while (n < 1 || n > 1000){
        std::cout << "Invalid input. Enter a number from 1 - 1000. ";
        return 0;
    }

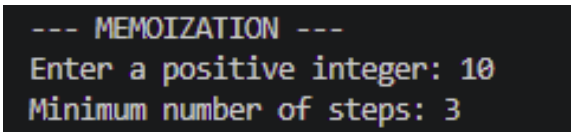
    for (int i = 0; i <= n; i++){
        memo[i] = -1;
    }

    std::cout << "Minimum number of steps: " << getMinSteps(n) << std::endl;

    return 0;

}
```

OUTPUT



ANALYSIS

The memoization code uses recursion to find the minimum number of steps needed to reach 1. It checks three possible operations: subtracting 1, dividing by 2, or dividing by 3 if the number is divisible by them. It uses std::min() to choose the operation with the fewest steps. The results are stored in the memo array so that the program does not need to calculate the same result again. This makes the code more efficient than using recursion alone. Its time and space complexity are both because it stores and calculates the results for numbers up to O(n).

Table 12-3. Bottom-Up Dynamic Programming Implementation

INPUT

bottomUpDp.cpp > ...

```cpp

#include <iostream>
#include <algorithm>

int getMinSteps(int n){
    int dp[1001], i;
    dp[1] = 0;
    for (i = 2; i <= n; i++){

        dp[i] = 1 + dp[i - 1];
        if (i % 2 == 0)
            dp[i] = std::min(dp[i], 1 + dp[i / 2]);
        if (i % 3 == 0)
            dp[i] = std::min(dp[i], 1 + dp[i / 3]);
    }

    return dp[n];
}

int main (){
    int n;

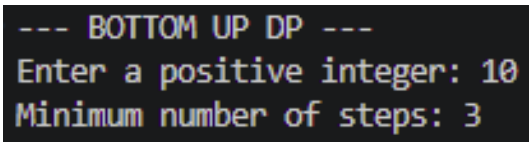
    std::cout << "--- BOTTOM UP DP ---" << std::endl;
    std::cout << "Enter a positive integer: ";
    std::cin >> n;

    if (n < 1 || n > 1000){
        std::cout << "Invalid input. Enter a number from 1 - 1000.";
        return 0;
    }

    std::cout << "Minimum number of steps: " << getMinSteps(n) << std::endl;

    return 0;
}
```

OUTPUT



ANALYSIS

The bottom-up code also finds the minimum number of steps needed to reach 1, but instead of using recursion, it uses a loop. It starts with dp[1] = 0 because no steps are needed to reach 1. Then, it calculates the minimum steps for each number from 2 up to the input number. It checks the possible operations and uses std::min() to choose the one with the fewest steps. The results are stored in the dp array and used to calculate the next numbers. Its time and space complexity are both O(n), and it avoids recursion by solving the problem in order.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

GRESOLA:

INPUT

SUPPLEMENTARY ACTIVITY > main.cpp > ...

```cpp
#include <iostream>

const int ROWS = 4;
const int COLS = 4;

int matrix[ROWS][COLS] ={
    {7, 4, 8, 5},
    {1, 3, 7, 6},
    {5, 3, 4, 6},
    {2, 2, 7, 2}
};

int targetCost;
int countPaths(int row, int col, int cost){
    if (row >= ROWS || col >= COLS) {
        return 0;
    }

    cost += matrix[row][col];

    if (cost > targetCost){
        return 0;
    }
    if (row == ROWS - 1 && col == COLS - 1){
        return (cost == targetCost) ? 1 : 0;
    }

    // where recursion happens:
    int rightPaths = countPaths(row, col + 1, cost);
    int downPaths = countPaths(row + 1, col, cost);
    return rightPaths + downPaths;
}

int main (){
    std::cout << "Matrix:" << std::endl;
    for (int i = 0; i < ROWS; i++){
        for (int j = 0; j < COLS; j++){
            std::cout << matrix[i][j] << "\t";
        }
        std::cout << std::endl;
    }

    std::cout << "Enter target cost: ";
    std::cin >> targetCost;
```

```cpp
45
46      if (targetCost < 0){
47          std::cout << "Target cost must be non-negative." << std::endl;
48          return 1;
49      }
50
51      int result = countPaths(0, 0, 0);
52      std::cout << "Target cost: " << targetCost << std::endl;
53      std::cout << "Number of valid paths: " << result << std::endl;
54
55      return 0;
56  }
```

OUTPUT | TARGET COST = 30

```
Matrix:
7       4       8       5
1       3       7       6
5       3       4       6
2       2       7       2
Enter target cost: 30
Target cost: 30
Number of valid paths: 2
```

```
Matrix:
7       4       8       5
1       3       7       6
5       3       4       6
2       2       7       2
Enter target cost: 20
Target cost: 20
Number of valid paths: 0
```

PSEUDOCODE

```
CONSTANT ROWS = 4
CONSTANT COLS = 4

MATRIX =
    7   4   8   5
    1   3   7   6
    5   3   4   6
    2   2   7   2


FUNCTION countPaths(row, col, cost):

    IF row >= ROWS OR col >= COLS:
        RETURN 0
    END IF

    cost = cost + MATRIX[row][col]

    IF cost > targetCost:
        RETURN 0
    END IF
```

```
    IF row == ROWS - 1 AND col == COLS - 1:
        IF cost == targetCost:
            RETURN 1
        ELSE:
            RETURN 0
        END IF
    END IF

    rightPaths = countPaths(row, col + 1, cost)
    downPaths = countPaths(row + 1, col, cost)

    RETURN rightPaths + downPaths

END FUNCTION


MAIN
START

    PRINT "Matrix:"

    FOR i = 0 TO ROWS - 1:
        FOR j = 0 TO COLS - 1:
            PRINT MATRIX[i][j]
        END FOR

        PRINT NEW LINE

    END FOR

    PRINT "Enter target cost:"
    INPUT targetCost

    IF targetCost < 0:
        PRINT "Target cost must be non-negative."
        STOP WITH ERROR CODE 1
    END IF

    result = countPaths(0, 0, 0)

    PRINT "Target cost:", targetCost
    PRINT "Number of valid paths:", result

END
```

ANALYSIS

I decided to create my own matrix to try the algorithm further because I noticed that there was also a mistake in the matrix provided in the example. In my code, the user can enter a target cost, and the program will count how many possible paths have that exact total cost. The countPaths() function uses recursion to check the paths by moving either right or down. It adds the value of each cell to the current cost and stops checking a path if the cost exceeds the target or goes outside the matrix. Once it reaches the last cell, it checks if the total cost matches the target. If it does, the path is counted as valid. After checking the possible paths, the program adds the results and prints the total number of valid paths.

DIZOR:

INPUT

HOA12.1dizor.cpp

```cpp
#include <iostream>

const int M = 4;
const int N = 4;

    //4x4 Matrix
int grid[M][N] = {
    {7, 4, 8, 5},
    {1, 3, 7, 6},
    {5, 3, 4, 6},
    {2, 2, 7, 2}
};

int findPaths(int r, int c, int current_sum, const int goal) {
    if (r >= M || c >= N) {
        return 0;
    }

    current_sum += grid[r][c];

    if (current_sum > goal) {
        return 0;
    }

    if (r == M - 1 && c == N - 1) {
        return (current_sum == goal) ? 1 : 0;
    }

    int r_ways = findPaths(r, c + 1, current_sum, goal);
    int d_ways = findPaths(r + 1, c, current_sum, goal);

    return r_ways + d_ways;
}
```

```cpp
int main() {
    std::cout << "Grid Layout:\n";
    for (const auto& row : grid) {
        for (int val : row) {
            std::cout << val << "\t";
        }
        std::cout << "\n";
    }

    int total_cost = 0;
    std::cout << "\nEnter desired target cost: ";
    std::cin >> total_cost;

    if (total_cost < 0) {
        std::cout << "Error: Target cost cannot be negative.\n";
        return 1;
    }

    int paths_found = findPaths(0, 0, 0, total_cost);

    std::cout << "\nTarget Cost: " << total_cost << "\n";
    std::cout << "Number of valid paths: " << paths_found << "\n";

    return 0;
}
```

OUTPUT

```
Grid Layout:
7       4       8       5
1       3       7       6
5       3       4       6
2       2       7       2

Enter desired target cost: 26

Target Cost: 26
Number of valid paths: 2
```

PSEUDOCODE

```
ALGORITHM findPaths(r, c, current_sum, goal)
    IF r >= M OR c >= N THEN
        RETURN 0
    END IF

    current_sum = current_sum + grid[r][c]

    IF current_sum > goal THEN
        RETURN 0
    END IF

    IF r == M - 1 AND c == N - 1 THEN
        IF current_sum == goal THEN
            RETURN 1
        ELSE
            RETURN 0
        END IF
    END IF

    r_ways = findPaths(r, c + 1, current_sum, goal)
    d_ways = findPaths(r + 1, c, current_sum, goal)

    RETURN r_ways + d_ways
END ALGORITHM

ALGORITHM main()
    PRINT "Grid Layout"
    FOREACH row IN grid DO
        FOREACH val IN row DO
            PRINT val with tab character
        END FOREACH
        PRINT newline
    END FOREACH

    total_cost = 0
    PRINT "Enter desired target cost: "
    READ total_cost

    IF total_cost < 0 THEN
        PRINT "Error: Target cost cannot be negative."
        RETURN 1
    END IF

    paths_found = findPaths(0, 0, 0, total_cost)
```

```
    PRINT "Target Cost: ", total_cost
    PRINT "Number of valid paths: ", paths_found
END ALGORITHM
```

ANALYSIS

It functions in a recursive brute force strategy to solve the path, exploring every possible valid route from the top left corner, to the bottom right corner on a 4x4 matrix. If row and column goes beyond grid limits, it stops the execution of that path branch instantly and returns 0. Because the grid contains positive integers, if current_sum surpasses the 'goal' early, it becomes impossible to reduce it back down and the code truncates the path early but returns 0.

CONCLUSION

GRESOLA:

Through this activity, I learned about different algorithmic strategies, such as recursion, brute force, backtracking, greedy algorithms, divide-and-conquer, and dynamic programming. I understood that each strategy has its own way of solving problems, and choosing the right approach depends on the task. Through the procedure, I learned how memoization and bottom-up approaches store and reuse results to solve problems more efficiently. In the supplementary activity, I applied recursion to count the possible paths in a matrix with a given cost, moving only to the right or downward. Overall, this activity helped me understand how different strategies can be applied to solve problems in a more organized and efficient way.

DIZOR:

Algorithmic strategies such as recursion, brute force, backtracking, greedy, divide and conquer strategy, helps to develop a programmatic solution to trace paths through a grid and show how these paradigms shift our understanding from manual calculations, to systematic, and scalable problem solving. Understanding these strategies help us predict how our program behave and as data scales, ensuring applications stay fast and responsive under heavy loads

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: ChatGPT

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Grammar correction, code assistance, and explanation of concepts.

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: We used ChatGPT for grammar correction, code assistance, and explanations of concepts we did not fully understand. It did not replace our own thinking, analysis, or completion of the required work.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.